Deep Learning & Transformers

From Perceptrons and Sequential Bottlenecks to Self-Attention, LLMs, and RAG

Author: Abhinav Gupta | **Email:** abhinavgupta15.ag@gmail.com | **Focus:** Theoretical Foundations & Applied Architectures

1. Foundations of Deep Learning

Deep Learning is a subset of Machine Learning focused on learning hierarchical representations of data through multi-layered artificial neural networks. Each successive layer extracts increasingly abstract features directly from raw data, eliminating the need for manual feature engineering.

Core Mathematical Formulation of a Dense Layer:

Feed-Forward Linear Transformation & Non-Linear Activation

$$y = \sigma(W \cdot x + b) \quad \text{where } W \in \mathbb{R}^{m \times d}, x \in \mathbb{R}^d, b \in \mathbb{R}^m$$

Key foundational components include:

- **Activation Functions:** Introduce non-linearity into network transformations (ReLU: max(0, z), GELU: x · Phi(x), Swish, Sigmoid, Softmax).
- **Loss Function & Optimization:** Quantify empirical error (Cross-Entropy, MSE) and optimize weights via Stochastic Gradient Descent (Adam, AdamW) with Backpropagation.
- **Convolutional Neural Networks (CNNs):** Exploit translation invariance and spatial locality through parameter sharing and pooling kernels for vision and grid-like data.
- **Recurrent Neural Networks (RNNs, LSTMs, GRUs):** Designed for sequential temporal sequences by maintaining a hidden memory state h_t across time steps.

2. The Sequential Bottleneck: Why Recurrent Models Failed

For years, sequence modeling relied on Recurrent Neural Networks (RNNs) and their gated variants (LSTMs and GRUs). In an RNN, the hidden state at step t is computed sequentially:

Recurrent Step Transition

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

Despite gating mechanisms in LSTMs, sequential models suffer from three fatal bottlenecks:

- **Inability to Parallelize (O(N) Compute Dependency):** The computation of step t depends strictly on step t-1. GPUs with thousands of parallel tensor cores sit largely idle waiting for previous time steps.
- **Vanishing and Exploding Gradients:** Gradients backpropagated through hundreds of time steps shrink exponentially toward zero or explode, severely limiting learning on long sequences.
- **Information Bottleneck:** Compressing an entire 500-word paragraph into a single fixed-size vector h_T leads to catastrophic loss of early token information.

3. The Transformer Paradigm Shift ('Attention Is All You Need', 2017)

In 2017, Vaswani et al. published *'Attention Is All You Need'*, introducing the **Transformer architecture**. The radical breakthrough was discarding recurrence entirely and replacing it with an all-to-all **Self-Attention mechanism**. Every token in a sequence can attend directly to every other token simultaneously in O(1) path length, unlocking massive hardware parallelism on modern GPUs.

Dimension	Recurrent Networks (RNN / LSTM)	Transformer Architecture
Compute Parallelism	Sequential O(N) — No GPU parallelism across time	Fully Parallel O(1) across all tokens simultaneously
Long-Range Path Length	O(N) sequential hops through hidden states	O(1) direct attention connections between any pair
Memory Complexity	O(N) state footprint	O(N^2) quadratic attention matrix (standard)
Gradient Flow	Prone to exponential decay across time steps	Direct highway via residual add & norm skips

4. Mathematical Architecture of the Transformer

A Transformer processes an input sequence of tokens represented by embedding matrix X in R^{N x d_{model}}. The architecture is composed of Self-Attention, Positional Encoding, and Feed-Forward Networks.

4.1 Scaled Dot-Product Attention

Tokens are projected into three distinct representations via learned linear projections: **Query (Q)** (what am I looking for?), **Key (K)** (what do I contain?), and **Value (V)** (what content do I offer?):

Linear Projections & Scaled Dot-Product Attention Equation

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V \qquad (W_Q, W_K \in \mathbb{R}^{d_{model} \times d_k}, W_V \in \mathbb{R}^{d_{model} \times d_v})$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

Why the scaling factor 1 / sqrt(d_k)?

For large embedding dimensions d_k, the dot products grow large in magnitude, pushing the softmax function into regions with extremely small gradients (vanishing gradient problem). Dividing by sqrt(d_k) normalizes the variance back to 1.0, preserving gradient health during backpropagation.

4.2 Multi-Head Attention (MHA)

Instead of computing a single attention function, Multi-Head Attention linearly projects queries, keys, and values h times into distinct subspaces of dimension d_k = d_{model} / h. This allows the model to simultaneously attend to information from different representation subspaces at different positions (e.g., syntactic structure in Head 1, entity resolution in Head 2, semantic mood in Head 3):

Multi-Head Attention Formulation

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot W_O$$

$$\text{where} \quad \text{head}_i = \text{Attention}(Q \cdot W_{Q,i}, \quad K \cdot W_{K,i}, \quad V \cdot W_{V,i})$$

4.3 Positional Encodings

Because attention is permutation-invariant (reordering tokens produces the exact same attention outputs), order information must be explicitly injected. Classic Transformers use fixed sinusoidal position functions:

Sinusoidal Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(pos / 10000^{2i / d_{model}} \right)$$

$$PE_{(pos, 2i+1)} = \cos\left(pos / 10000^{2i / d_{model}} \right)$$

Modern LLMs (LLaMA, Mistral) employ **Rotary Position Embeddings (RoPE)**, which apply a 2D rotation matrix directly to Q and K vectors, naturally incorporating relative positional distances through complex inner products.

4.4 Position-Wise Feed-Forward Network (FFN) & LayerNorm

Each attention layer is followed by a two-layer MLP applied identically and independently to each position: FFN(x) = max(0, x · W_1 + b_1) · W_2 + b_2. Modern architectures replace ReLU with **SwiGLU**. Residual connections with **Pre-Layer Normalization (Pre-LN)** or **RMSNorm** ensure stable gradient flow across hundreds of stacked transformer layers.

5. Taxonomy of Transformer Architectures

Category	Attention Masking	Representative Models	Primary Use Cases
Encoder-Only	Bidirectional (Tokens attend to both past and future)	BERT, RoBERTa, DeBERTa, BGE-Embeddings	Text Classification, Token Tagging, Dense Embeddings
Decoder-Only	Causal / Autoregressive (Tokens attend ONLY to prior tokens)	GPT-4, LLaMA-3, Mistral, DeepSeek-V3	Text Generation, Conversational AI, Reasoning, Code
Encoder-Decoder	Bidirectional Encoder + Causal Cross-Attention Decoder	T5, BART, FLAN-T5	Language Translation, Abstractive Summarization

6. Breakthrough Innovations in Modern Transformers

Scaling Transformers to trillions of parameters and million-token contexts required groundbreaking efficiency innovations:

- **FlashAttention (IO-Aware Exact Attention):** Traditional attention materializes the intermediate N x N attention matrix into slow GPU High-Bandwidth Memory (HBM). FlashAttention fuses operations using tiling and online softmax, computing attention within ultra-fast on-chip SRAM with zero memory overhead.
- **Grouped-Query Attention (GQA) & Multi-Query Attention (MQA):** Instead of maintaining unique Key and Value heads for every Query head, GQA shares K and V heads across groups of Q heads. This drastically shrinks the KV-Cache footprint during autoregressive generation by 4x to 8x with zero loss in generation quality.
- **KV Caching:** During next-token generation, previously computed Key and Value tensors are saved in memory so past tokens are not recomputed, transforming generation cost per step from O(N^2) to O(N).
- **Mixture of Experts (MoE):** Replaces dense FFN layers with multiple sparse expert networks (e.g. 8 or 64 experts). A lightweight router activates only top-k (e.g. 2) experts per token, allowing massive parameter capacity (e.g. 671B params) while computing with only active compute (e.g. 37B params).

7. Role of Transformers in Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation combines external factual knowledge retrieval with generative LLMs. Transformers power every stage of modern RAG pipelines:

> ***Key Takeaway for RAG:*** *LLMs suffer from parametric memory decay, hallucination, and outdated knowledge. RAG decouples memory storage (Vector Database populated with Transformer Embeddings) from reasoning capacity (Decoder-Only LLM).*

1. Dense Vector Embeddings (Bi-Encoder Transformers):

Encoder models (such as BAAI/bge-small-en, OpenAI text-embedding-3) transform text chunks into semantic vector spaces. By pooling the final hidden states (e.g., mean pooling or [CLS] token), texts with semantic similarity are clustered close together in vector space.

2. Cross-Encoder Re-Ranking:

While bi-encoders generate vectors independently for fast ANN search, cross-encoders feed (Query, Document) pairs simultaneously into full cross-attention layers, computing precise relevance scores to filter top retrieved chunks.

3. Context Injection & Generative Synthesis:

The retrieved passages are injected into the Decoder-Only LLM's prompt context. Using Causal Self-Attention, the model grounds its answers strictly in the retrieved evidence, minimizing hallucinations and enabling citations.

> ***Summary & Future Outlook:*** *Deep Learning has transformed from rigid task-specific architectures into general-purpose Transformer-based foundation models. Understanding the mathematical mechanics of Q, K, V attention, positional dynamics, and embedding spaces provides the fundamental bedrock for engineering production-grade GenAI, Agentic Workflows, and RAG architectures.*